- *Dependency analysis*—It analyzes prop and state dependencies to determine when components truly need to re-render, preventing unnecessary computations.
- *Transparent optimization*—Developers can continue writing idiomatic, declarative React code without needing to sprinkle optimization logic throughout the codebase.

During the build process, the React Compiler integrates seamlessly with existing tooling like Babel and ESLint, applying optimizations behind the scenes without altering the developer experience. This leads to faster, more efficient applications while preserving React's declarative programming style.

4.2 Data flow and state management

A solid understanding of data flow and state management is critical for building maintainable and performant React applications—and for demonstrating technical depth in interviews. React's declarative model is built around one-way data binding, where data flows predictably from parent to child, and state management, which keeps UI and application logic in sync. Mastering these concepts allows developers to design scalable architectures and handle complex application state effectively.

For Jane, grasping these principles not only boosted her confidence in interviews but also empowered her to design production-grade React apps with clean, maintainable data flows. Table 4.5 highlights key features of React's data flow and state management model, from foundational concepts to advanced techniques.

Table 4.5 Data flow in React

Topic	Fundamentals	Applied skills	Advanced
One-way data binding	Explain unidirectional flow.	Manage props, lifting state up.	Architect Context API or state container patterns for global state.